

Material de Consulta: Matplotlib Básico - Gráficos Simples

Introdução ao Matplotlib

O que é Matplotlib?

Matplotlib é a biblioteca de visualização de dados mais popular do Python. Pense nela como uma "caneta digital" que permite transformar números em gráficos visuais para:

- Análise exploratória de dados
- Apresentação de resultados
- Comunicação de insights
- Validação de hipóteses

Por que aprender Matplotlib?

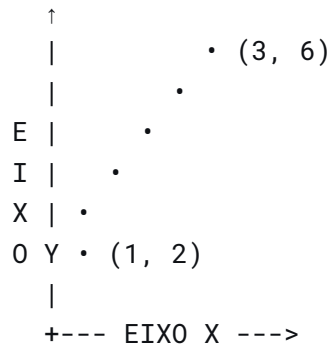
1. Versátil: Cria mais de 30 tipos de gráficos diferentes
 2. Personalizável: Controle total sobre cada elemento visual
 3. Integração: Funciona perfeitamente com NumPy, Pandas e outras bibliotecas
 4. Profissional: Padrão da indústria para visualização científica
-

Conceitos Fundamentais

A Estrutura de um Gráfico

text

TÍTULO



LEGENDA: Série A Série B

Tipos Básicos que Aprendemos:

1. Gráfico de linha (`plt.plot()`): Mostra tendências e mudanças ao longo do tempo
 2. Gráfico de dispersão (`plt.scatter()`): Mostra relação entre duas variáveis
-

Glossário de Atributos e Funções

1. Configurações Básicas do Gráfico

Função	Exemplo	Descrição	Valores Comuns
<code>title()</code>	<code>plt.title("Vendas")</code>	Define título principal	String com texto
<code>xlabel()</code>	<code>plt.xlabel("Meses")</code>	Nomeia eixo horizontal	String com texto
<code>ylabel()</code>	<code>plt.ylabel("R\$")</code>	Nomeia eixo vertical	String com texto
<code>xlim()</code>	<code>plt.xlim(0, 10)</code>	Limites do eixo X	(mínimo, máximo)
<code>ylim()</code>	<code>plt.ylim(0, 100)</code>	Limites do eixo Y	(mínimo, máximo)
<code>grid()</code>	<code>plt.grid(True)</code>	Ativa/desativa grade	<code>True</code> ou <code>False</code>
<code>legend()</code>	<code>plt.legend()</code>	Mostra legenda	Chame sem parâmetros
<code>show()</code>	<code>plt.show()</code>	Exibe o gráfico	Sempre ao final

2. Atributos de Linha (para `plot()`)

Parâmetro	Exemplo	Descrição	Valores Comuns
<code>color</code>	<code>color='red'</code>	Cor da linha	Nomes: <code>'red'</code> , <code>'blue'</code> , <code>'green'</code> Códigos: <code>'#FF5733'</code>
<code>linewidth</code>	<code>linewidth=2.5</code>	Espessura da linha	Números: <code>1</code> , <code>2</code> , <code>3</code>
<code>linestyle</code>	<code>linestyle='--'</code> '	Estilo da linha	<code>'-'</code> (sólida) <code>'--'</code> (tracejada) <code>':'</code> (pontilhada) <code>'-.'</code> (traço-ponto)
<code>marker</code>	<code>marker='o'</code>	Forma dos pontos	<code>'o'</code> círculo <code>'s'</code> quadrado <code>'^'</code> triângulo <code>'D'</code> diamante
<code>markersize</code>	<code>markersize=8</code>	Tamanho dos pontos	Números: <code>4</code> , <code>8</code> , <code>12</code>
<code>label</code>	<code>label='2023'</code>	Texto para legenda	String descritiva

3. Atributos de Dispersão (para `scatter()`)

Parâmetro	Exemplo	Descrição
-----------	---------	-----------

<code>c</code>	<code>c='red'</code>	Cor dos pontos
<code>s</code>	<code>s=50</code>	Tamanho dos pontos
<code>alpha</code>	<code>alpha=0.5</code>	Transparência (0-1)
<code>edgecolors</code>	<code>edgecolors='black'</code>	Cor da borda

4. Cores Pré-definidas Úteis

```
python

# Cores básicas
'blue', 'red', 'green', 'orange', 'purple', 'brown', 'pink', 'gray'

# Tons de cinza (para impressão)
'black', 'dimgray', 'gray', 'lightgray', 'white'

# Código hexadecimal (16 milhões de cores!)
'#FF5733' # Laranja avermelhado
'#33FF57' # Verde limão

'#3357FF' # Azul vibrante
```

Exemplos Práticos com Explicação

Exemplo 1: Gráfico de Linha Simples

```
python

import matplotlib.pyplot as plt

# Dados: temperatura média por mês
meses = ['Jan', 'Fev', 'Mar', 'Abr', 'Mai', 'Jun']
```

```

temperatura = [28, 27, 25, 22, 19, 17]

plt.plot(meses, temperatura,
         color='darkred',      # Cor vermelha escura
         marker='o',          # Bolinhas nos pontos
         linewidth=2,          # Linha mais grossa
         linestyle='-',        # Linha contínua
         label='Temperatura') # Para legenda

plt.title('Temperatura Média - Primeiro Semestre')
plt.xlabel('Meses')
plt.ylabel('Temperatura (°C)')
plt.grid(True, alpha=0.3) # alpha = transparência da grade
plt.legend()

plt.show()

```

Exemplo 2: Gráfico de Dispersão

```

python

# Dados: relação horas de estudo vs nota
horas_estudo = [2, 5, 1, 8, 4, 6, 3, 7, 9, 5]
notas = [5.0, 6.5, 3.5, 8.5, 6.0, 7.5, 5.5, 8.0, 9.0, 7.0]

plt.scatter(horas_estudo, notas,
            c='darkblue',      # Cor dos pontos
            s=80,              # Tamanho dos pontos
            alpha=0.7,         # 70% opaco
            edgecolors='black') # Borda preta

plt.title('Relação: Horas de Estudo vs Nota')
plt.xlabel('Horas de Estudo (por semana)')
plt.ylabel('Nota Final')
plt.grid(True, linestyle=':') # Grade pontilhada

plt.show()

```

Exemplo 3: Múltiplas Séries

```
python

# Dados comparativos
trimestres = ['T1', 'T2', 'T3', 'T4']
vendas_2022 = [150, 180, 220, 190]
vendas_2023 = [170, 210, 240, 220]

# Primeira série
plt.plot(trimestres, vendas_2022,
         color='blue',
         marker='o',
         label='2022')

# Segunda série
plt.plot(trimestres, vendas_2023,
         color='red',
         marker='s',
         linestyle='--',
         label='2023')

plt.title('Vendas Anuais por Trimestre')
plt.xlabel('Trimestres')
plt.ylabel('Vendas (milhares)')
plt.legend()
plt.grid(True)

plt.show()
```

Boas Práticas - Checklist

✓ SEMPRE FAÇA:

1. Título descritivo - Responda: "O que este gráfico mostra?"
2. python

```
# ✗ RUIM:
plt.title("Gráfico 1")
```

 BOM:

3. `plt.title("Crescimento de Vendas 2020-2023")`
4. Rótulos dos eixos com unidades - Inclua medidas quando aplicável
5. python

 RUIM:

```
plt.ylabel("Valores")
```

 BOM:

```
plt.ylabel("Temperatura (°C)")
```

6. `plt.xlabel("Tempo (dias)")`
7. Escala apropriada - Não distorça os dados
8. python

Se seus dados vão de 0 a 100:

```
plt.ylim(0, 100) #  Justo
```

9. `plt.ylim(0, 200)` #  Cuidado: pode minimizar variações
10. Legenda clara - Quando tiver múltiplas séries
11. python

```
plt.plot(x, y1, label='Grupo A') # Nome descritivo  
plt.plot(x, y2, label='Grupo B')
```

12. `plt.legend()` # Não esqueça de chamar!

RECOMENDAÇÕES:

5. Cores com contraste - Garanta que sejam distinguíveis
6. python

 Bom contraste:

```
cores = ['darkblue', 'orange', 'green']
```

⚠ Evite:

7. cores = ['lightblue', 'lightcyan', 'skyblue'] # Tudo muito parecido
8. Grade sutil - Ajuda na leitura sem poluir visualmente
9. python
10. plt.grid(True, alpha=0.3) # Grade transparente
11. Tamanho adequado - Ajuste conforme necessidade
12. python
13. plt.figure(figsize=(10, 6)) # Largura=10", Altura=6"
14. Salve em alta qualidade - Para apresentações e relatórios
15. python

```
plt.savefig('meu_grafico.png', dpi=300, bbox_inches='tight')  
# dpi = pontos por polegada (resolução)
```

16. # bbox_inches='tight' = remove bordas em branco

❌ EVITE:

9. Cores vibrantes demais - Podem ofuscar
10. Muitas séries - Limite a 3-4 no mesmo gráfico
11. Marcadores muito grandes - Podem sobrepor dados
12. Títulos muito longos - Seja conciso

Solução de Problemas Comuns

Problema 1: Gráfico não aparece

python

```
# ❌ Falta plt.show()  
plt.plot(x, y)  
# plt.show() # Descomente esta linha!
```

✅ Correto:

```
plt.plot(x, y)
```

```
plt.show()
```

Problema 2: Gráfico sobreposto ao anterior

```
python
```

```
# ❌ Gráficos se misturam
plt.plot(x, y1)
plt.plot(x, y2) # Sobreposto ao primeiro

# ✅ Limpe entre gráficos
plt.figure() # Cria uma nova figura vazia
plt.plot(x, y1)
plt.show()

plt.figure() # Outra nova figura
plt.plot(x, y2)

plt.show()
```

Problema 3: Legenda não aparece

```
python
```

```
# ❌ Falta label= nos dados
plt.plot(x, y) # Sem label
plt.legend() # Legenda vazia

# ✅ Adicione labels
plt.plot(x, y, label='Minha Série')

plt.legend()
```

Problema 4: Textos cortados

```
python
```

```
# ❌ Títulos ou legendas podem cortar
plt.tight_layout() # ✅ Adicione esta linha antes de plt.show()
```

```
plt.show()
```

Template Básico para Reutilizar

```
python
```

```
import matplotlib.pyplot as plt

# 1. DADOS
x = [1, 2, 3, 4, 5]
y = [10, 15, 13, 17, 20]

# 2. CONFIGURAÇÃO INICIAL (opcional)
plt.figure(figsize=(8, 5)) # Tamanho personalizado

# 3. CRIAR GRÁFICO
plt.plot(x, y,
         color='#2E86AB',    # Cor hexadecimal
         linewidth=2,
         marker='o',
         markersize=8,
         label='Dados')

# 4. PERSONALIZAR
plt.title('Título Descritivo Aqui', fontsize=14)
plt.xlabel('Nome do Eixo X (unidade)', fontsize=12)
plt.ylabel('Nome do Eixo Y (unidade)', fontsize=12)
plt.grid(True, alpha=0.3, linestyle='--')
plt.legend()

# 5. AJUSTES FINAIS
plt.tight_layout() # Ajusta espaçamento

# 6. SALVAR (opcional, antes de show())
plt.savefig('meu_grafico.png', dpi=300)

# 7. EXIBIR

plt.show()
```

Próximos Passos

Depois de dominar gráficos simples, você pode explorar:

1. Gráficos de barras - Comparação entre categorias
2. Histogramas - Distribuição de frequências
3. Subplots - Múltiplos gráficos na mesma figura
4. Estilos pré-definidos - `plt.style.use('ggplot')`

Dica Final:

```
python

# Descubra todos os estilos disponíveis
print(plt.style.available)
# Experimente:
plt.style.use('ggplot') # Estilo parecido com R
plt.plot(x, y)

plt.show()
```

Lembrete Importante:

"Um bom gráfico conta uma história com clareza. Comece sempre perguntando: 'Qual história meus dados precisam contar?' e então escolha o gráfico e as cores que melhor comunicam essa mensagem."

Bons estudos! 🚀